Draw It or Lose It

CS 230 Project Software Design Template

Version 1.0

Table of Contents

Document Revision History

Version	Date	Author	Comments
2.0	06/07/2026	Huston Alger	Submission for Project Two

Instructions
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

The Gaming Room has approached and requested a software design solution for their game “Draw It or Lose It”, Which is now solely an android app and will be a Web based game where teams compete to guess images that are rendered on screen. To support this the software must have multiple teams, multiple players per team, as well as unique naming for all entities involved, and strict controls to ensure that there are not multiple games existing in memory at one time but just one game at a time within the memory at any given time.

The design proposed currently uses proven development techniques to meet its requirements, however a centralized game service will allow management over all game data as well as enforce a single instance rule using the singleton pattern. This guarantees consistency across the gaming and software environment. The software will also be utilizing an iterator pattern which verifies unique games and team names to prevent any duplication. The application has structuring built on the OOP or object oriented principles that work through a shared entity base class. All together having these techniques and patterns being utilized within the software create scalable and maintainable architecture that is platform ready and supports the clients current and future needs.

Requirements

Design Constraints

Developing this game “Draw It or Lose It” as a web-based application that supports multiple platforms and users of the game simultaneously raises several constraints to keep an eye out for. Because of the way the game is intended on being run across multiple platforms with many users the architecture for it needs to be scalable, Light and have the capability to handle coinciding access without having issues to the performance. This limits the resources of intensive components and requires careful management of the shared data to prevent any conflicts between the team & player dynamic.

The larger constraint is with the requirement that only one game can exist within the memory at any given time, which affects how this application must be structured. This requires a singleton pattern which will centralize management over the game, maintaining consistency. The other constraint that comes to mind is having the requirement for unique names for both teams and games. This would mean the system must have reliable searching and comparison logic within it, which in turn affects how collections are stored and accessed. The iterator pattern would be necessary to allow a checking for any duplications before any new entities are added.

System Architecture View

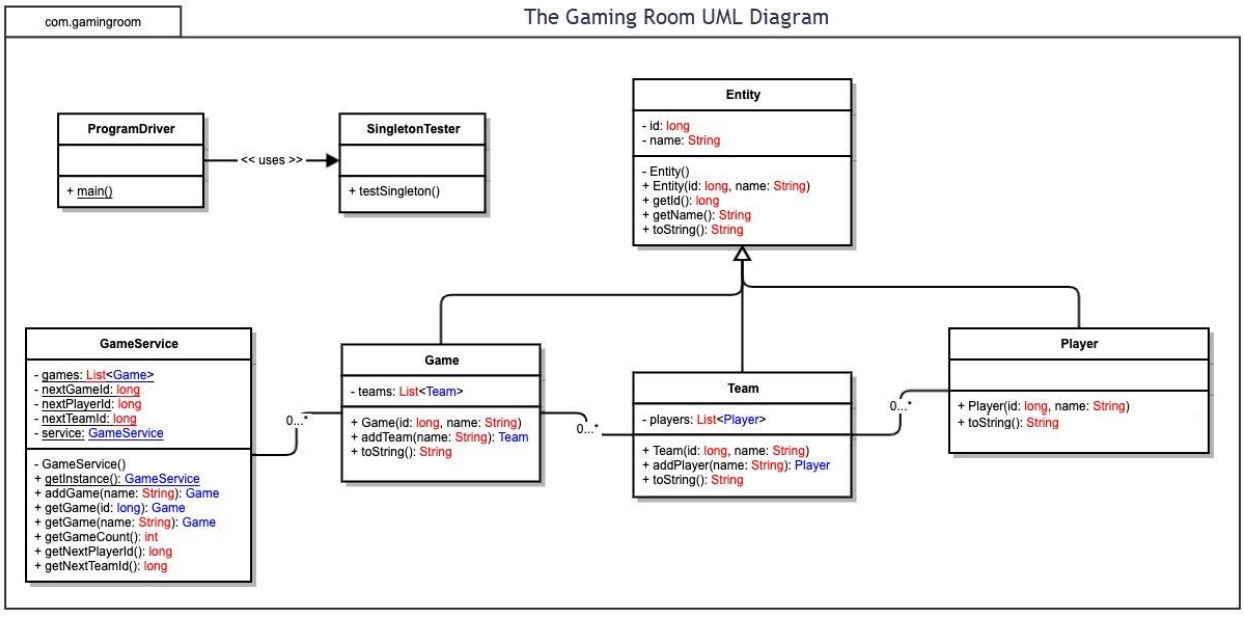
Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The UML diagram for “Draw It or Lose it” definitively shows the core structuring of the application itself. It demonstrates how the main components of the application interact with each other to meet the requirements needed. The base of the model or base class is the Entity class which provides the attributes of “id”and “name” as well as common behaviors including “getid()”, “getName()”, and “toString()”. As for the rest of the classes such as Game, Team, and Player, well they all each inherit from the Entity class. This demonstrates the OOP or Object Oriented Principles of inheritance. The design allows and ensures that all of the game related objects are following a consistent structure for “id” and “name” or naming. This design is also necessary in that it will eliminate any redundancy within it.

You can see within the diagram that between the classes represent composition relationships between them. Looking at the Game class and Team class, you can see that Game has a list of team objects and each of the Team(s) contain a list of player objects. The relationship being represented supports the clients requirements that the game has to allow multiple teams, and within those teams there must be multiple players. In the GameService class you can see it manages all of the game instances and maintains a list or lists of games, unique ids for not just players but also teams and games. This is done using a singleton pattern, which ensures only one game exists within the memory at any time, Which meets the clients requirements for the single active game instance.

You will also see in the diagram that the use of the iterator pattern in GameService, Game, and Team is used to search through collections and enforces the requirement for unique names before any new entities can be added. The UML demonstrates OOP, encapsulation, as well as controlled access. It represents a scalable and maintainable design which aligns with the clients requirements for the software.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined

below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	**Mac**	**Linux**	**Windows**	**Mobile Devices**

Server Side	Mac systems can host any and most web based apps with the help of tools like: Nginx, Apache, or Node.js. However MacOS isn't a commonly known OS to be used as a production server platform. MAC is better for a development environment than a hosting solution for many users. So While MacOS doesn't require an separate OS licensing, The hardware offered by Apple is expensive and not a go to used in could or data center environments. For the gaming room MacOS would present operational limitations as well as unnecessary costs, which makes it a less practical choice for draw it or lose it.	Linux is one of if not the strongest and widely used platforms for hosting web based applications. Not only does it support major web servers and frameworks such as python, Apache, and others, but it also is known for stability, scalability and excellent security. Linux or most pf their distributions are free/open source, which offers a low licensing cost, very low, as well as offering (optional) enterprise support. For the gaming room you can see Linux offers a cost effective, reliable and easily/highly scalable environment capable of supporting thousands of players. This would be an ideal server for the game.	The Windows server offers strong administrative tools, while being able to host web based applications when utilizing Microsoft technologies such as IIs or ASP.net. The only drawback is Windows requires paid licensing and may require a client Access License, or a CALs. This would increase costs as the application or release grows. While being a very solid option for organization, it is a more expensive option when compared to Linux. For the Gaming Room windows is viable but cost heavy.	Mobile devices like IOS/IPhone, and Android are incapable of hosting a larger scale web based application. Their main functionality rests in the clients connecting to the server, They influence how the server must handle the large amount of concurrent connection, mobile devices are not capable of hosting server side applications. While there are no licensing costs with mobile devices within the server side, the limitations shine when the need for reinforcing the need for a centralized scalable sever platform such as Linux or Windows to support the game.

Client Side	In supporting Mac clients, their involvement means making sure the web app runs smoothly with the modern MacOS browsers such as Safari or chrome or even Firefox. The development team has to account for these browser specific behaviors and differences in rendering which will most likely add testing time. Where there are no extra licensing costs for supporting the Mac users, that extra comes into the developers time when cross browser testing and responsive design. Time is the cost in this scenario to make sure the interface behaves and performs across browsers consistently. Having experience in HTML5, Javascript and cross platform web development is enough to support the Mac clients.	Linux users will have easy access to the game through the standard compliant browsers like Firefox or chrome. These browsers follow the modern web specifications very closely. This again makes Linux easier to support from the clients perspective. There are no licensing fees or costs associated with the Clients using Linux, but again the development team still has to complete testing across Linux distributions to make sure the applications behaves correctly across the board. So the main things to focus on are: Browser compatibility, responsive design, and correct and effective web interface performance. Only requirement needed would be front end development expertise.	While windows is and remains one one of the most used and adopted desktop platforms, supporting it would require thorough testing across multiple browsers. Considering each browser handles certain JavaScript or Css differently the team will have to ensure the responsive interface is consistently working across all of the different browsers. No additional licensing costs for supporting windows clients but the investment of time for this because of its large user base would be more substantial. Having knowledge in cross browser debugging and responsive web designing is vital to ensure windows users have a good and smooth experience.	Mobile devices will need the most attention by far. The application has to be fully responsive and optimized to fit smaller screens, touch inputs and a variety of conditions that come along with mobile devices. Andriod users will most likely access the game through Chrome, While Ios users will be using safari. The development team will have to work to ensure the adaptable interface works with different screen sizes, and performance constraints. Again no extra licensing fees or costs for Mobile devices but the need for expertise and a good amount of time for the mobile design will be necessary. Knowing and understanding mobile design, cross device testing and performance optimization will be needed to ensure the usability and responsiveness of the game on phones and tablets.

Development Tools	Mac Is a strong developmental environment for both mobile and web software. The developers can use tools such as: JetBrains IDE, Eclipse, and Visual Studio coding when it comes to the web application. Mac also includes Xcode, which is a requirement for any IOS development, which makes mac essential if the Gaming Room wants to expand past browser based mobile support. The majority of tools on MacOS are free or low cost, with the exception of certain IDES. The impact on the development team would be a positive one because MacOS can support a wide range of languages and frameworks while offering a stable environment for front and back end development.	Linux offers a more flexible and more powerful development environments for web based applications, Supporting a wide range of languages like python, Java, PHP, which in turn go with tools like Visual studios or eclipse. Linux can also integrate naturally with any containerization tools like Docker and orchestration platforms. These are commonly used for scalable web deployments not development. Most Linux tools are free and open source which in turn keeps licensing costs low for the development team. The environment Linux provides is a consistent one that will align with production servers which in turn will reduce compatibility issues and improve efficiency/work flow.	Windows has a broad network of developmental tools like: Visual studio + (code), Java script, Jetbrains IDE, and Java. The visual studio community is free and well equipped for web projects, although some third party tools may require paid licensing. Windows is strong for any .Net and C development which makes it a solid option if Back end is utilizing ASP.Net. The platform will also support Node.js and Java as well as others which allows the team more flexibility. The impact on the development team is a positive one , because of the fact Windows offers a familiar and well run & supported environment, the only downside being the cost could be higher depending on the tools utilized.	For Mobile development tools, well those will be dependent on the platform. Android development will use Android studio, whereas IOS requires Xcode on the MacOS. While both of these tools are free, The Gaming Room or Draw It or Lose It will need to run as a responsive web app inside both mobile browsers. The tools needed for this would be front end development technologies such as JavaScript, CSS, or HTML5. The developers will need to maintain and rely on the browser developmental tools, real device testing and emulators to make sure the compatibility is smooth across different devices, screen sizes and operating systems. The licensing costs are somewhat minimal however the time the team must invest into this will be hefty to ensure its success.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: Based on the overall evaluation of all the platforms, Linux seems and is the most appropriate operating platform for expanding Draw It or Lose It into a scalable web-based, and multi-platform environment. Linux offers over the top performance, stability, and security while also being cost-effective because of its open-source licensing. It's widely used in cloud and enterprise environments which makes it ideal for hosting a high-traffic game server that must support thousands of concurrent players. Its compatibility with modern web technologies and server frameworks makes Linux the strongest and the most future-proof choice for The Gaming Room.

2. **Operating Systems Architectures**: Again, the recommended Linux architecture follows a standard multi-tier web application model which: separates the presentation layer, application logic, and data storage. This architecture supports a horizontal scaling, allowing additional server instances to be added as the player base grows. Linux also integrates well with containerization technologies such as Docker and orchestration tools like Kubernetes, which enables: efficient deployment, load balancing, and fault tolerance. This architecture ensures that the game will remain responsive, reliable, and maintainable as new features and platforms are added.

3. **Storage Management**: A relational database management system or (RDBMS) such as MySQL or PostgreSQL is recommended for storage management. These systems run efficiently on Linux and provide a strong support for the structured data, relationships, and transactional integrity which is important for managing players, teams, game states, amongst others. They also support replication and clustering, which in turn improves performance and reliability. This storage approach ensures that game data remains consistent, secure, and accessible even under heavy load.

4. **Memory Management**: Linux utilizes a highly efficient memory management system that includes: virtual memory, caching, and process isolation. This is most beneficial for Draw It or Lose It because the game requires a strict control over a single active game instance in memory. The server can allocate dedicated memory to the game service while using the caching to speed up repeated operations when it comes to things such as loading drawings or validating team names. Linux's memory management will make sure that the application remains stable, prevents memory leaks, and maintains performance even as the number of connected clients increases.

5. **Distributed Systems and Networks**: Supporting the communication between various platforms the game will operate as a distributed web application using standard internet protocols; The server will expose RESTful or WebSocket-based APIs that will allow clients on Windows, Mac, Linux, Android, or iOS to communicate with the central game service. The Load balancers and redundant server instances can be used to maintain availability during any outages or spikes within the traffic. This approach ensures that all of the clients will receive real-time updates and

that the system remains resilient even if or when individual components fail or lose connectivity.

6. **Security**: Security is very essential for protecting user information and maintaining their trust. Linux provides a strong built-in security feature(s) such as: user permissions, firewalls, and secure shell access. I believe the application should utilize the HTTPS to encrypt all of the communication between the clients and the server, which will prevent any unauthorized access or data interception. Some other additional Secuirty such as input validation, authentication tokens, and secure session management can help safeguard user accounts and game data. By combining Linux security capabilities with secure coding practices of the development team, The Gaming Room can ensure that Draw It or Lose It remains safe across all supported platforms.